

FeelsLike

Buildings that listen.

A digital-twin building optimizer with natural-language feedback: occupants complain in plain language, a parser turns the complaint into a typed constraint, and a constraint-aware HVAC controller acts on a physics-based thermal simulation — cutting energy while keeping every occupied minute inside the comfort band.

-26.6%
HVAC energy vs the static schedule
buildings run today (7 simulated
days)

0 min
comfort violations — vs 16,328 min
for the baseline over the same week

50
case NLP benchmark with a held-out
split the parser was never tuned on

2M
PPO training steps on the twin;
ablation-tested against 3 baselines

Contents

1	The problem & the idea
2	System architecture
3	The digital twin: 5-zone RC thermal model
4	Understanding complaints: the NLP parser
5	The constraint engine: decay, arbitration, memory
6	Controllers & the 7-day A/B result
7	The reinforcement-learning track
8	NLP benchmark: methodology & honest numbers
9	The live demo system
10	Scale story: money & carbon
11	Limitations & path to production
12	Anticipated questions
A	Repository map & commands

1 · The problem & the idea

Commercial buildings spend roughly 40–50% of their electricity on HVAC, and most of them run it deaf: a facilities team sets a fixed schedule — typically an aggressive setpoint like 22 °C from early morning to late evening — precisely *because* nobody can hear the occupants. Overcooling everyone, everywhere, all day is the cheapest way to avoid complaints when feedback arrives days late through a ticket system, if it arrives at all. The result is the familiar picture: sweaters and space heaters under desks in August, while the chiller runs flat out.

The feedback that could fix this already exists. People say "it's stuffy in here" out loud, in chat, constantly. It just never reaches the control loop.

FeelsLike closes that loop. A complaint typed in Slack or a web chat ("it's really stuffy in Conference Room B") is parsed into a typed, schema-validated constraint, the constraint steers a zone-level HVAC controller, and a physics-based digital twin proves the effect: the building relaxes its blanket overcooling everywhere people *aren't* uncomfortable, and spends energy exactly where they are.

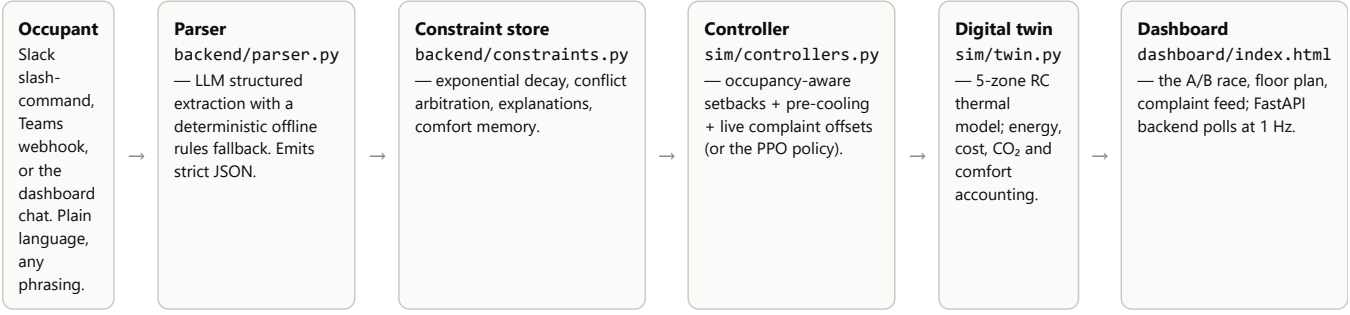
The one-line result: over seven simulated days on identical weather, FeelsLike used **26.6% less energy** than the static schedule with **zero** occupied minutes outside the comfort band — while the obvious alternative, simply raising the thermostat, saved slightly more (31.7%) but broke comfort for 429 minutes. Efficiency *without sacrifice* is the point, and both halves of that claim are measured, not asserted.

Why this is not "just a thermostat app"

- **It listens in natural language** — sarcasm, typos, and Hinglish included — with a benchmarked parser and explicit anti-hallucination guardrails.
- **It arbitrates disagreement.** Two people in one room asking for opposite things get a severity-weighted, time-decaying compromise with a human-readable explanation — not a war over the thermostat.
- **It proves the effect.** Two identical twins run in lock-step on the same weather, one listening and one not. Every number shown is that A/B difference.
- **It degrades gracefully.** No API key, no Wi-Fi, no problem: an offline rules parser keeps the entire system functional.

2 · System architecture

Everything communicates through one contract, agreed at hour zero and never renegotiated: a complaint becomes a `ParsedComplaint`, which becomes a `Constraint`, which nudges a controller acting on the twin.



The contract

```
{ "is_comfort_complaint": true,
  "zone_id": "zone_b",           // null if no known zone was named
  "issue": "too_hot" | "too_cold" | "stuffy" | "humid" | "drafty" | "other",
  "severity": 1..3,              // 1 mild, 2 clear discomfort, 3 urgent
  "confidence": 0.0..1.0,
  "reasoning": "one short sentence" }
```

Guardrails that prevent judge-visible failures

- A `zone_id` outside the known zone list is **nulled** — the system never acts on a hallucinated zone.
- `zone_id = null` triggers a clarifying question, never a guess.
- Non-thermal messages ("the projector is broken") are classified `is_comfort_complaint = false` and explicitly ignored.
- Retractions ("it's fine now in Room B") **clear** the zone's constraints instead of filing a new complaint — including trap phrasings like "the heat issue is fixed now".
- Every parse is Pydantic-validated; anything malformed falls back to the rules parser.

Two twins always run in lock-step on identical weather — `FeelsLike` vs the Static-22 °C baseline. The dashboard's racing meters, savings percentages, and comfort counters are all that live A/B difference; nothing is precomputed or staged.

3 · The digital twin: 5-zone RC thermal model

The twin is a standard lumped-parameter (resistor–capacitor) building model — the same family used in building-simulation literature — chosen so its behavior is defensible and its parameters are physically interpretable. Each zone i integrates:

```
C_i · dT_i/dt = UA_i (T_out - T_i)           // envelope conduction
                + Σ_j G_ij (T_j - T_i)         // inter-zone coupling (shared walls)
                + Q_solar_i + Q_occupants_i     // gains: facade sun + 100 W/person
                + Q_hvac_i                     // cooling (negative)
```

HVAC is a "perfect thermostat with finite capacity": each 60 s step it removes exactly the heat needed to reach the setpoint, capped at the unit's capacity. Electrical power = thermal cooling / COP (3.4) + ventilation fan power (0 / 150 / 420 W per zone by fan level). Ventilation also adds 45 W/K of outdoor-air coupling per level — fresh air costs energy when it's hot outside, which is exactly the stuffy-vs-efficient trade-off the controller must navigate.

The building

ZONE	FACADE	AREA	C (J/K)	UA (W/K)	PEAK SOLAR	COOLING CAP	OCCUPANCY PROFILE
Open Office A	N	220 m ²	3.2 × 10 ⁶	500	1,600 W	16 kW	office: up to 24 people, 8:00–20:00
Conference Room B	S	60 m ²	0.9 × 10 ⁶	160	2,400 W	7 kW	meetings 10:00, 14:00, 16:30
Cabin C	E	40 m ²	0.6 × 10 ⁶	110	1,700 W	4 kW	3 people, 9:00–19:00
Lobby D	W	90 m ²	1.4 × 10 ⁶	300	2,600 W	8 kW	4 people, 8:00–20:00
Cafeteria E	S	110 m ²	1.6 × 10 ⁶	340	2,100 W	10 kW	lunch surge: 30 people 12:00–14:30

Inter-zone conductances (shared walls): A–B 90, A–C 70, A–D 110, D–E 90, B–C 40 W/K.

Weather, solar, occupancy

Weather is a seeded synthetic model of an Indian city in August: base 30 °C with a deterministic ±1.5 °C per-day wobble, a 6 °C diurnal swing (minimum ~3 AM, maximum ~3 PM), and a small smooth intra-day perturbation. Deterministic seeding is what makes the A/B race fair — every controller sees byte-identical weather. Solar gain follows each facade's orientation (east peaks 9:00, south 12:30, west 16:00; north gets diffuse only). Occupancy follows weekday/weekend piecewise schedules per zone — the lunch surge in the cafeteria and the 14:00 conference meeting are the recurring events the comfort-memory feature learns. An Open-Meteo hook exists to swap in real hourly weather with no interface change.

Comfort accounting

The occupied comfort band is 23.0–26.5 °C (ASHRAE-style for offices). Every simulated minute a zone is occupied and outside the band counts as a violation minute, with degree-minutes tracked above and below separately. Unoccupied zones accrue nothing — comfort is a promise to people, not to empty rooms. Energy is priced at ₹9/kWh (commercial ToU average) and 0.71 kg CO₂/kWh (India grid, CEA ~FY23).

4 • Understanding complaints: the NLP parser

The parser's job is deceptively narrow — one chat message in, one JSON constraint out — but the input space is human: sarcasm ("great, another sauna day 🙄"), Hinglish ("cabin mein bahut garmi hai bhai"), typos, indirect phrasing ("I'm wearing a jacket"), and messages that merely *mention* a room without complaining about it.

Two parsers, one interface

Primary — LLM structured extraction. A system prompt carries the schema, the known zone list with aliases, explicit anti-hallucination rules, and worked examples (including Hinglish and the projector-is-broken negative). Any OpenAI-compatible provider works via `LLM_BASE_URL`: Anthropic Claude, OpenAI, or free tiers — Groq (Llama 3.3 70B, sub-second), Google AI Studio (Gemini 2.5 Flash), or a fully local Ollama model. The demo runs on Groq's free tier at zero cost.

Fallback — deterministic rules. Keyword-cascade parser: most-specific issue first (drafty before cold, so "cold air blowing on me" is a draft, not a temperature complaint), zone resolution via aliases plus "room B"-style regex, severity from intensity words. It is instant, offline, and free — the demo-day insurance. The switch is automatic: any LLM error, timeout, or missing key degrades gracefully, and the UI labels every message with its parse source (`llm` or `rules`) and latency honestly.

Retraction handling

"It's fine now, thanks" must *cancel* a constraint, not file one — and "the heat issue in the lobby is fixed now" must not be read as a heat complaint just because it contains the word "heat". A retraction detector (all-clear phrases, guarded by ongoing-markers like "but" / "still") runs before keyword matching; on retraction with a known zone, the zone's active constraints are expired immediately and the feed shows an all-clear. This was a known failure mode we designed for rather than disclosed away.

Latency

Rules: ~0 ms. Groq free tier: typically 0.3–0.8 s. Claude/OpenAI: 1–2 s. The complaint → visible action loop on the dashboard is under a second in the offline path and 1–2 s end-to-end with a hosted LLM — fast enough that the parse feels like a reflex on stage.

5 · The constraint engine: decay, arbitration, memory

Complaints decay — recency-weighted democracy, not a ticket queue

A complaint's influence decays exponentially with a **45-minute half-life** and expires at 2 hours unless re-reported:

$$\begin{aligned} \text{decay}(t) &= 0.5^{(\text{age} / 45 \text{ min})} && (\emptyset \text{ after } 2 \text{ h}) \\ \text{weight}(t) &= \text{severity} \times \text{confidence} \times \text{decay}(t) \end{aligned}$$

This single design choice solves three problems at once: stale complaints don't haunt the building forever, a persistent problem naturally re-asserts itself (people complain again), and no manual "close ticket" step is ever needed.

Effects

ISSUE	SETPOINT OFFSET (SEV 1/2/3)	AIRFLOW
too_hot	−0.8 / −1.3 / −1.8 °C	—
too_cold	+0.8 / +1.3 / +1.8 °C	—
stuffy	−0.3 / −0.4 / −0.5 °C	fan +1
humid	−0.4 / −0.5 / −0.6 °C	fan +1
drafty	+0.3 / +0.4 / +0.5 °C	fan −1

Conflict arbitration

Opposing constraints in one zone resolve by **weighted mean** — weight = severity × confidence × decay — with a human-readable explanation generated for the dashboard and the Slack reply: *"CONFLICT in zone: stuffy (sev 2, 3m ago) vs too cold (sev 2, 1m ago) → severity-weighted compromise: −0.1 °C."* Neither party wins forever, because both claims decay. Every office has this argument; FeelsLike settles it transparently in seconds.

Comfort memory (learning the building's habits)

The store keeps complaint history, and a pattern miner clusters it by (zone, issue, hour-of-day). When the same complaint has occurred on ≥ 2 distinct days at a similar hour (±1 h), the system pre-applies a gentle constraint (severity 1, confidence 0.5) **30 minutes before** the pattern's usual time and announces it in the feed: *"Conference Room B usually reports stuffy around 14:00 — pre-applying a gentle fix."* The building stops waiting to be told twice. Memory only learns from real occupant complaints, never from its own pre-applies, so it cannot self-reinforce.

6 · Controllers & the 7-day A/B result

CONTROLLER	WHAT IT MODELS
Static 22 °C schedule	What buildings do today: overcool everything on a fixed schedule (07–21 h weekdays, 08–18 h weekends), fans always on. The complaint-avoidance strategy of a deaf building.
Reactive thermostat 24 °C	The obvious "fix": per-zone thermostat at a warmer setpoint, only when occupied. Saves more energy — and breaks comfort, because it reacts to the current minute only.
FeelsLike (constraint-aware)	Occupancy-aware setbacks (25 °C occupied / 26 °C pre-cool 30 min ahead / 28.5 °C unoccupied), crowd-aware fans, plus the live complaint offsets from §5. The demo controller.
FeelsLike (PPO agent)	A learned policy that offsets the rules controller's setpoints ± 2 °C. §7.

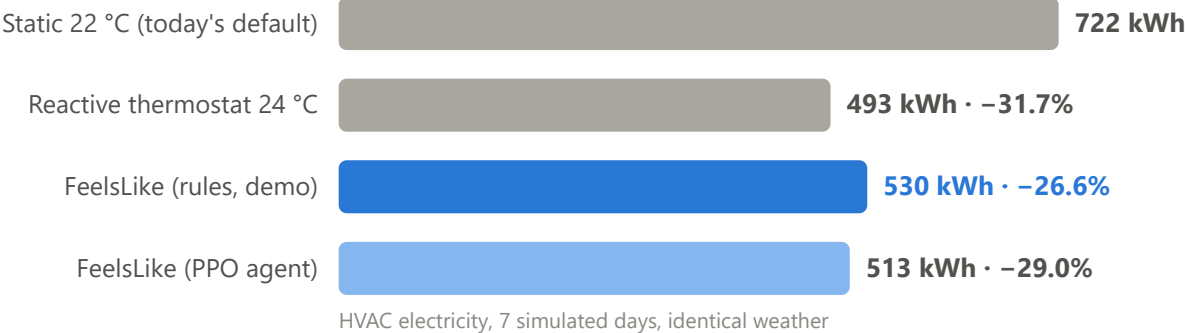


Figure 1 — Energy: 7 simulated days, identical weather (seed 7). Lower is better.



Figure 2 — Comfort: occupied minutes outside the 23–26.5 °C band over the same week. The baseline's 16,328 min is systematic overcooling; the reactive thermostat's 429 min is the cost of naive efficiency.

CONTROLLER	KWH	₹	KG CO ₂	VIOL-MIN	HOT °-MIN	COLD °-MIN	SAVED
Static 22 °C schedule	722.4	6,502	512.9	16,328	418	15,973	—
Reactive thermostat 24 °C	493.4	4,441	350.3	429	1,281	0	31.7%
FeelsLike (rules)	530.3	4,773	376.5	0	0	0	26.6%
FeelsLike (PPO)	512.7	4,614	364.0	22	3	0	29.0%

The reactive thermostat is the argument, not the embarrassment. It saves 5 points more energy than FeelsLike — and pays 429 minutes of discomfort for it. Anyone can save energy by letting rooms drift; the hard part is saving energy while keeping the comfort promise. FeelsLike's zero holds because pre-cooling starts 30 minutes before people arrive and complaints steer the controller before drift becomes suffering.

7 · The reinforcement-learning track

Environment

SPECIFICATION	
Episode	1 simulated weekday; control decision every 5 minutes (288 steps)
Observation (18-d)	5 zone temps · outdoor temp · sin/cos hour-of-day · 5 zone occupancies · 5 active complaint offsets
Action (5-d, continuous)	per-zone setpoint offset in $[-2, +2]$ °C <i>around the rules controller's decision</i> — the agent learns residual corrections, not raw control
Reward	$-\Delta\text{kWh} - 0.12 \times \Delta(\text{degree-minutes outside band}) - 0.6 \times \text{unmet-complaint pressure}$
Complaints in training	0–3 synthetic complaints injected at random times/zones per episode, so the policy learns to respect human constraints, not just the static band
Algorithm	PPO (stable-baselines3), MlpPolicy, 8 parallel envs, n_steps 512, batch 1024, lr 3×10^{-4} , γ 0.995, 2M steps

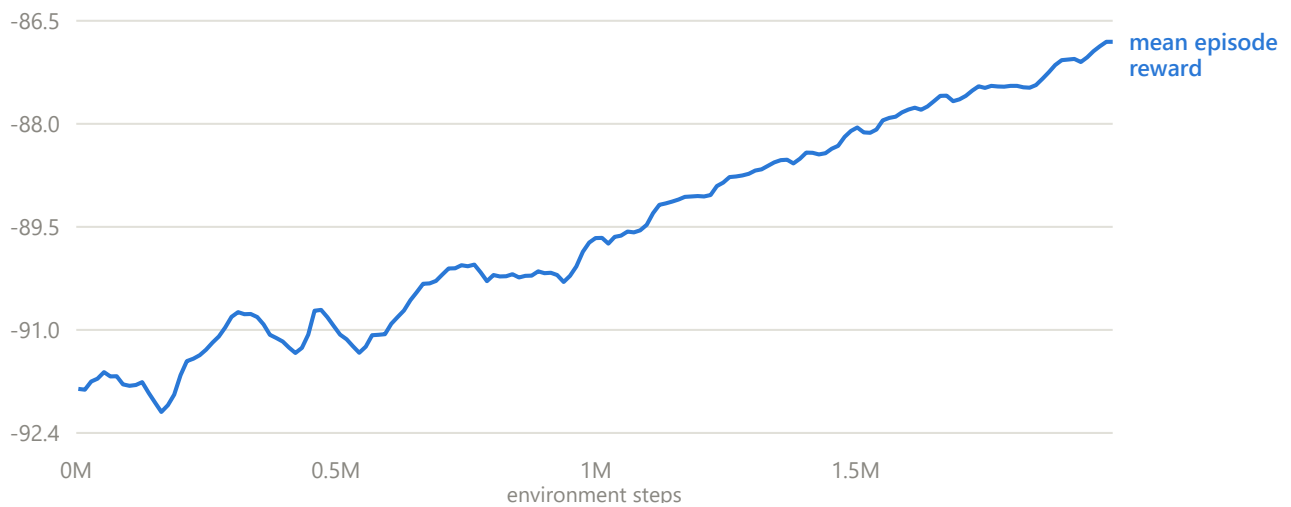


Figure 3 — Learning curve: mean episode reward over 2M environment steps. Steady improvement ($-91.8 \rightarrow -86.8$), no plateau — the policy is still learning when the budget ends.

Ablation (10 random weekdays, mean $\pm$ sd)

CONTROLLER	KWH MEAN	KWH SD	VIOL-MIN MEAN
Static 22 °C schedule	111.1	8.3	2,831
Reactive thermostat 24 °C	82.3	6.5	79
FeelsLike (rules)	84.4	8.1	0
FeelsLike (PPO)	81.6	8.3	2

The honest decision: PPO beats the rules controller on energy (-2.8 kWh/day, and -29.0% vs -26.6% on the 7-day run) but averages 2 violation-minutes per day against the rules controller's zero. Our acceptance bar was "wins on energy at equal-or-better comfort" — PPO doesn't clear it yet, so **the demo ships the rules controller** and presents RL as a trajectory. The learning curve is still climbing at 2M steps; a longer run may flip the decision. We would rather show a zero than 1.9 kWh.

8 · NLP benchmark: methodology & honest numbers

The benchmark is 50 labeled cases scoring three things per message: complaint detection (is this a comfort complaint at all?), zone extraction, and issue extraction. "Exact triple" requires all three correct.

The split is the methodology. The 30-case *dev* set is what the rules parser was tuned on — its 100% there is table stakes, not evidence. The 20-case *held-out* set was written after the rules were frozen and never used for tuning: typos ("its friezing in the confrence room"), sarcasm ("love how I need gloves to type"), Hinglish variants, multi-zone messages, retractions with keyword traps, and negatives designed to fool keyword matching ("the coffee machine is steaming hot again"). Held-out numbers are the honest ones.

METRIC	RULES · DEV	RULES · HELD-OUT	LLM · DEV	LLM · HELD-OUT
Complaint detection	30/30 (100%)	12/20 (60%)	30/30 (100%)	15/20 (75%)
Zone extraction	30/30 (100%)	18/20 (90%)	30/30 (100%)	17/20 (85%)
Issue extraction	30/30 (100%)	14/20 (70%)	30/30 (100%)	16/20 (80%)
Exact triple	30/30 (100%)	11/20 (55%)	30/30 (100%)	12/20 (60%)

LLM = Llama 3.3 70B via Groq free tier through the schema-validated prompt; identical guardrails on both parsers.

What still fails (held-out examples, shown as measured)

- "open office AC ki hawa direct desk pe aa rahi hai" — Hinglish draft phrasing; both parsers miss drafty.
- "love how I need gloves to type in the open office" — pure sarcasm with no temperature keyword; both parsers miss the complaint.
- "did you see how hot it is outside today? 40 degrees" — weather small-talk; the LLM (and rules) still read it as a heat complaint.
- "both the lobby and the cafeteria are boiling" — multi-zone; the schema carries one zone, a disclosed structural limitation.

The gap between 100% (dev) and ~55–60% (held-out) is precisely **why the LLM parser is the product and the rules are the insurance**: the rules ceiling on unseen phrasing is structural, while the LLM path improves with model quality at zero code change. We report the gap rather than hiding the held-out set because a benchmark you can't fail isn't a benchmark.

9 · The live demo system

Dashboard

A single-file, zero-dependency dashboard (inline CSS/JS, no build step, nothing to break offline) polling `/api/state` at 1 Hz:

- **The race** — paired racing bars (FeelsLike vs baseline kWh) with the live saved-% as the headline, and the comfort-violations counter beside it: both halves of the claim on one screen.
- **Floor plan** — the real 5-zone geometry, color-mapped to temperature, with setpoints, occupancy, fan levels, complaint offsets, pulsing constraint markers, and an amber conflict ring when a zone is under arbitration. Unoccupied zones render dimmed so overnight drift doesn't read as failure.
- **Occupant channel** — chat with quick-chips that are the demo script (the conflict pair, Hinglish, and a deliberate non-complaint), each message showing its parse, source badge, and latency.
- **Resilience UX** — typed messages survive failed sends; a reconnecting pill appears if the backend dies; the parser badge flips to `rules (offline fallback)` honestly if the LLM path fails mid-demo.

Slack / Teams

POST `/api/slack` accepts Slack slash-command form posts and Teams-style JSON. A complaint typed in Slack changes a zone on the dashboard in seconds, and the bot replies in-channel with the parsed action and explanation. The web chat is the same endpoint — the fallback if venue Wi-Fi dislikes webhooks.

The 3-minute script

T	BEAT
0:00	Hook: buildings are deaf, so they overcool everyone "just in case". HVAC $\approx$ 40–50% of commercial electricity.
0:20	Live: "it's really stuffy in Conference Room B" → parsed chip, zone shifts, latency badge.
1:00	Teammate: "Room B is freezing, I'm wearing a jacket" → CONFLICT badge, weighted compromise, explanation on screen.
1:30	Crank to 960×: racing bars open the gap over a simulated week — 26.6% less energy, zero violations, and the reactive counter-example.
2:20	Money/carbon tiles; scale story (\$10).
2:45	Close: "Buildings have been deaf for a hundred years. We taught one to listen." Hand judges the card: type anything.

10 · Scale story: money & carbon

The simulated building is ~520 m². Its measured week — 192 kWh saved, ₹1,729, 136 kg CO₂ — is small on purpose: it's one honest building. The point is the mechanism scales linearly with floor area and requires **no new hardware**: complaints already come from tools offices use, and the control outputs are ordinary setpoint/fan commands any BMS accepts.

	THIS BUILDING (MEASURED, 7 DAYS)	10,000 M ² OFFICE (EXTRAPOLATED, ANNUAL)
Energy saved	192 kWh	~192 MWh
Cost saved	₹1,729	~₹18–22 lakh
CO ₂ avoided	136 kg	~160 tonnes

Extrapolation assumes similar climate, occupancy patterns, and the same 26.6% saving rate; it is a scale illustration, not a measurement — and labeled as such wherever we show it.

11 · Limitations & path to production

Known limitations (disclosed, not discovered)

- **"In simulation."** The zero-violation result holds because HVAC sizing is adequate in the model. Real buildings have undersized units and thermal surprises; the claim on stage is always "in simulation".
- **Parser held-out ceiling.** ~55–60% exact-triple on adversarial held-out phrasing (§8). Multi-zone complaints carry one zone. Both are benchmarked and shown.
- **RL not yet shipping.** The PPO agent trades 2 viol-min for 2.8 kWh/day; below our bar (§7).
- **Synthetic weather/occupancy.** Deterministic by design for fair A/B; the Open-Meteo hook and BMS-log calibration are the path to real data.

Path to production

- **Same architecture, real actuators:** the twin swaps for BACnet/Modbus writes; the constraint engine and parser are unchanged.
- **Calibration:** fit each zone's R/C parameters from a week of BMS logs — a standard system-identification step for lumped-parameter models.
- **Privacy by construction:** constraints aggregate at zone level; no identity is needed or stored beyond a display name in the feed.
- **Safety envelope:** setpoints are clamped to 21.5–29 °C and offsets cap at ± 1.8 °C — the building can be steered, never hijacked.
- **RL flip-in:** the PPO policy is a drop-in controller class; when a longer run clears the comfort bar, one line switches the demo to it.

12 · Anticipated questions

QUESTION	ANSWER
How do you stop the LLM hallucinating a setpoint?	It never emits one. The LLM only classifies into a schema; unknown zones are nulled, low confidence asks a clarifying question, non-complaints are ignored. Control math is deterministic code.
Is the RL real?	2M-step PPO with learning curves and a 4-way ablation — and we're showing you the run where it <i>didn't</i> beat our bar, which is how you know the numbers are real.
Two people disagree?	Live demo: severity × confidence × recency weighted compromise with a printed explanation. It decays, so neither wins forever.
How real is the twin?	Standard lumped-parameter RC model (literature-backed) with zone coupling, solar, occupancy, and capacity-limited HVAC. Calibration path: fit R/C from a week of BMS logs.
What if the Wi-Fi dies on stage?	It can — the parser auto-falls back to offline rules and the badge says so. The demo was designed to survive its own failure modes.
Why not just raise the thermostat?	We benchmarked exactly that (Reactive 24 °C): more savings, 429 minutes of broken comfort. That trade-off is the product's reason to exist.
Privacy?	Zone-level aggregation; no identity required.

Appendix A · Repository map & commands

PATH	WHAT IT IS
sim/twin.py	5-zone RC thermal digital twin (physics, energy, comfort accounting)
sim/weather.py	Seeded synthetic weather + facade solar + Open-Meteo hook
sim/controllers.py	Static, reactive, constraint-aware, and PPO controllers
sim/env.py	Gymnasium environment (obs/action/reward, synthetic complaint injection)
backend/parser.py	LLM + rules complaint parser, retraction detection, provider config
backend/prompts.py	The structured-extraction system prompt
backend/constraints.py	Decaying constraint store, arbitration, explanations
backend/memory.py	Comfort memory: recurring-pattern mining and pre-application
backend/app.py	FastAPI: lock-step A/B sim, complaint API, Slack/Teams webhook, dashboard
dashboard/index.html	Single-file live dashboard (race, floor plan, occupant channel)
evals/	50-case benchmark (dev/held-out splits), runner, result JSONs
rl/train.py · rl/evaluate.py	PPO training (VecMonitor curves) and ablation
scripts/demo_day.py	7-day controller comparison → results JSON (the evidence table)
scripts/build_report.py	This document

Reproduce every number in this report

```
python -m scripts.demo_day           # Figures 1-2 and the §6 table
python -m evals.run_nlp_eval --rules  # §8 rules columns (offline)
python -m evals.run_nlp_eval         # §8 LLM columns (needs a key; free via Groq)
python -m rl.train --steps 2000000    # Figure 3 (rl/models/progress.csv)
python -m rl.evaluate --episodes 10   # §7 ablation table
uvicorn backend.app:app --reload      # the live demo, http://127.0.0.1:8000
```